



Formation Kubernetes

Session 5 — Secrets : Les Bases et le Problème

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session



Le problème

Où sont vos secrets ? Pourquoi c'est dangereux

10 min



K8s Secrets natifs

Comment ils fonctionnent, limites et risques

10 min



Panorama des solutions

Sealed Secrets, External Secrets, CSI, Vault

10 min



CSI Secret Store Driver

Architecture et workflow avec GCP Secret Manager

15 min



Démo + TP

Setup et mounting de secrets via CSI en action

45 min

Recap — Session 4



PersistentVolume

Stockage persistant découplé du pod — NFS, GCP disques



PersistentVolumeClaim

Requête de stockage — interface entre app et infra



StatefulSet

Déploiement avec identités stables — idéal pour les bases de données



PostgreSQL en prod

Cloud SQL sur GCP — backup auto, scaling, haute disponibilité

Question critique : comment l'API accède-t-elle à la BDD ? Où est le password ?

Le problème : où sont vos secrets ?

```
apiVersion: v1
kind: Deployment
metadata:
  name: api
spec:
  template:
    spec:
      containers:
      - name: api
        env:
        - name: DB_PASSWORD
          value: "P@ssw0rd!"
```



Git history

Visible à tous les devs



kubectl get pods

Expose les env vars



CI/CD logs

Logs publiques ou cachées

⚠ base64 ≠ chiffrement | C'est juste de l'encodage — n'importe qui peut décoder

K8s Secrets natifs – Fonctionnement

Créer un Secret

```
kubectl create secret generic  
  my-secret \  
  --from-literal=  
  password=secret123
```

Stocké dans etcd

```
apiVersion: v1  
kind: Secret  
data:  
  password: c2VjcmlV0  
# c2VjcmlV0 = base64
```

Usage dans un Pod (montage en volume ou env var)

```
env:  
- name: DB_PASSWORD  
  valueFrom:  
    secretKeyRef:  
      name: my-secret  
      key: password
```

✓ Mieux que du hardcoding | ✗ Pas de chiffrement natif | ✗ Lisible dans etcd en clair

Panorama des solutions

Sealed Secrets

- Chiffre K8s Secrets
- Clé asymétrique par cluster
- Facile à déployer

Limité à K8s — pas multi-cloud

External Secrets Operator

- Sync depuis Vault, AWS, GCP
- Source de vérité externe
- Rotation automatique

Opérateur K8s — complexe

CSI Secret Store

- Mount comme volume
- Cloud-native (GCP, AWS)
- Zéro Secret K8s

Notre approche chez RFLKT

HashiCorp Vault

- Solution complète
- Audit trail détaillé
- Multi-cloud & on-prem

Complexe et lourd

Nous utilisons CSI Secret Store Driver avec GCP Secret Manager en production

CSI Secret Store Driver — Architecture



1

volumeMount

Pod monte un volume 'secrets'

2

SecretProviderClass

Spécifie QUELS secrets, du QUEL provider

3

CSI Driver

Kubelet appelle le driver → lookup GCP Secret Manager

4

Fetch secret

GCP retourne le secret (si Workload Identity ok)

5

Mount comme fichier

Secret monté en /var/run/secrets/... — fichier décrypté

Workload Identity

Pod ← GCP Service Account

Service Account ← IAM Role

IAM Role ← Accès Secret Manager

Zéro clé statique — tout basé sur l'identité du Pod

CSI Secret Store : Multi-cloud

Aspect	GCP	AWS	Azure
Secret Store natif	Secret Manager	Secrets Manager	Key Vault
CSI Provider	Secrets Store CSI	Secrets Store CSI	Secrets Store CSI
Workload Identity	Workload Identity	IRSA (IAM Roles)	AAD Pod Identity
CRD K8s	SecretProviderClass	SecretProviderClass	SecretProviderClass
Configuration	resourceName: projects/.../ secrets/...	arn:aws:secretsmanager:...	vaultName, objectName
Audit Trail	Cloud Logging	CloudTrail	Azure Audit Logs

Le pattern CSI est le MÊME pour tous les clouds — seule la configuration change

La progression du TP – Sécurité progressive



Sécurité



Risques résiduels

- ✗ Git history
- ✗ kubectl get
- ✗ Docker image

- ✓ Git
- ✗ kubectl get
- ✓ Docker

- ✓ Git
- ✓ kubectl get
- ✓ Docker

Architecture du TP

exercices namespace

```
Pod: api-<NOM>
├─ ServiceAccount: training-apps
├─ volumeMount: /mnt/secrets-store
└─ CSI volume: secrets-store

SecretProviderClass: gcp-secrets-<NOM>
├─ provider: gcp
└─ secrets: [training-api-key]
```



GCP Secret Manager

```
Project: cloud-447406

Secret: training-api-key
├─ Version: 1
└─ Value: api-key-12345

IAM Permission
└─ training-apps@ →
    roles/secretmanager.secretAccessor
```

Étapes du TP

1. Créer Secret
GCP

2. SecretProvider
Class YAML

3. Pod avec
CSI volume

4. Vérifier
mount

5. Lire secret
comme fichier

Démo live — secret_manager_csi en production

Terraform: secret_manager_csi

Module Setup:

- └ CSI Driver install (Helm)
- └ GCP Service Account
- └ Workload Identity binding

Secrets Mapping:

- └ staging-app-db-password
- └ staging-app-api-key
- └ staging-mcp-server-linear
- └ ... (multi-app)



Production RFLKT

Environments:

- └ staging
- └ production

Secret Naming:

{env}-{app}-{key}

All Apps:

- └ app (API)
- └ mcp-server
- └ ... (via locals)

Récapitulatif et mini-challenge

4 points clés à retenir



base64 $\neq$ chiffrement

K8s Secrets sont exposés dans etcd — pas sûr en prod



CSI Secret Store Driver

Pas de Secret K8s — secret monté en fichier depuis le cloud



Workload Identity

Pod identifié → IAM → accès granulaire aux secrets



Audit trail

Chaque accès au secret loggué dans GCP Logs



Mini-challenge TP

Après le montage du secret DB : créez une SecretProviderClass supplémentaire pour un 2e secret (API key)